

JavaScript 함수 심화

학습 체크리스트

- ☐ var 변수 선언과 function 선언문의 호이스팅 오작동 메커니즘 및 런타임 사이드 이펙트 인지 여부
- ☐ 함수 스코프와 블록 스코프의 변수 유효범위 차이 및 let/const를 통한 스코프 통제 여부
- ☐ 렉시컬 스코프(정적 스코프)의 함수 선언 위치 기준 상위 스코프 결정 방식 이해 여부
- ☐ 클로저(Closure)의 정의와 외부 함수 종료 후에도 상위 렉시컬 환경의 변수를 유지하고 은닉하는 원리 파악 여부

호이스팅

호이스팅 (Hoisting)

자바스크립트 엔진이 코드를 실행하기 전, 스코프 내의 변수 및 함수 선언을 최상단으로 끌어올려 메모리에 선제 등록하는 현상

- 초기화 전 등록: 변수와 함수가 물리적 선언 위치보다 먼저 스코프에 등록됨
- 실행 전 단계: 코드가 순차 실행(런타임)되기 전에 파싱(컴파일) 단계에서 수집됨
- 두 가지 줄기: 변수 선언 방식(`var`)과 함수 선언 방식(`function`)에 따라 내부 동작이 다름

1. var 변수 선언의 호이스팅

- 동작 특징: 선언 단계와 초기화 단계가 분리 없이 동시에 진행되며 `undefined` 로 초기화됨
- 위험한 참조: 선언 코드보다 앞선 시점에서 변수를 조회해도 에러 대신 `undefined` 를 반환함
- 중복 허용: 동일 스코프 내에서 변수를 여러 번 선언해도 덮어쓰며 경고가 뜨지 않음
- 사이드 이펙트: 논리적 선후 관계를 붕괴시키고 기존 값을 강제 파괴하는 원인이 됨

var 호이스팅 코드 예시

```
1 // 1. 선언 이전 호출 시 에러 없이 undefined 출력 (오작동 우려)
2 console.log(score);
3
4 var score = 100;
5 console.log(score); // 100
6
7 // 2. 중복 선언 허용으로 기존 데이터가 경고 없이 파괴됨
8 var score = 200;
9 console.log(score); // 200
```

2. function 선언문의 호이스팅

- 동작 특징: 함수 선언문은 단순 초기화를 넘어 **함수 본체 전체**가 통째로 메모리에 로드됨
- 호출 시점 유연성: 물리적 선언 코드보다 상단에서 함수를 호출해도 정상 구동함
- 오버라이트 위험:
 - 서로 다른 스크립트에서 우연히 같은 이름의 함수를 정의할 경우 경고 없이 덮어쓰워짐
 - 런타임에 어떤 함수가 실행될지 통제하기 어려워 대규모 협업에서 버그의 온상이 됨
- 극복 방안: `let` 이나 `const` 에 익명 함수를 대입하는 **함수 표현식**으로 정의함

함수 호이스팅 코드 예시

```
1  // 1. 선언 이전에 함수 실행 가능 (구조적 가독성을 해침)
2  console.log(calculate(10)); // 20
3
4  function calculate(n) {
5      return n * 2;
6  }
7
8  // 2. 동일한 이름의 함수 중복 선언 시 아무 제재 없이 덮어씌워짐
9  function calculate(n) {
10     return n + 100;
11 }
12 console.log(calculate(10)); // 110 (이전 calculate 함수 소실)
```

스코프

스코프 (Scope)

식별자(변수명, 함수명 등)가 선언된 위치에 따라 유효하게 참조되고 영향력을 행사할 수 있는 한계 범위

- **참조 한계:** 선언된 영역 바깥에서는 식별자에 접근할 수 없도록 격리함
- **이름 충돌 방지:** 스코프가 분리되어 있으면 서로 다른 영역에서 동일한 이름의 식별자 사용 가능
- **유형:** 전통적인 함수 스코프(`var`)와 모던한 블록 스코프(`let`, `const`)로 구분됨

1. 함수 스코프 vs 블록 스코프

- 함수 스코프 (Function Scope)

- `var` 식별자가 따르는 유효 범위로, 오직 함수 블록(`function`)만 경계선으로 인정함
- 관통 현상: `if` 문, `for` 문, `while` 문 등의 일반 블록은 뚫고 밖으로 새어 나감

- 블록 스코프 (Block Scope)

- `let` 과 `const` 식별자가 따르는 유효 범위
- 모든 중괄호 블록(`{ }`, `if`, `for` 등) 내부로 식별자의 참조 한계를 완벽히 구속함

스코프 관통 및 격리 코드 예시

```
1  if (true) {
2    var globalA = "var 관통"; // 블록을 무시하고 탈출
3    let blockB = "let 격리";   // 블록 내부에 완전 감금
4  }
5
6  // 1. var 변수는 블록 밖에서도 안전하게 (?) 출력됨
7  console.log(globalA); // "var 관통"
8
9  // 2. let 변수는 블록 외부 접근 시 즉시 예외 차단
10 // console.log(blockB); // ReferenceError: blockB is not defined
```

2. 렉시컬 스코프

렉시컬 스코프 (Lexical Scope)

함수가 어디서 호출되었는지가 아니라, 최초에 어디서 선언(정의)되었는지에 따라 상위 스코프를 정적으로 결정하는 방식

- **정적 스코프 (Static Scope):** 런타임 호출 환경의 흐름에 지배받지 않고 물리적 코드가 작성된 시점에 변함없이 결정됨
- **렉시컬 환경 저장:** 함수는 생성 시점에 자신의 내부 슬롯에 선언 위치 기준의 상위 스코프 환경을 굳건히 저장함

렉시컬 스코프 코드 예시

```
1  const value = "Global";
2
3  // printValue는 전역(Global Scope)에서 선언됨
4  function printValue() {
5      // 선언 위치의 상위 스코프인 전역 value를 정적으로 굳건히 참조
6      console.log(value);
7  }
8
9  function execute() {
10     const value = "Local";
11     printValue(); // execute 내부에서 호출해도 결과는 "Global" 출력
12 }
13
14 execute();
```

클로저

클로저 (Closure)

함수가 선언된 시점의 주변 환경(렉시컬 환경)을 기억하여, 자신을 감싸는 외부 함수가 완전히 종료되어 실행 컨텍스트가 소멸한 후에도 상위 스코프의 변수에 계속해서 접근하고 제어할 수 있는 자바스크립트의 특수 결합체

- **환경의 보존:** 외부 함수의 실행 스택(Call Stack)이 비워진 뒤에도 끈질기게 변수 링크를 유지함
- **자유 변수 (Free Variable):** 클로저 함수에 의해 소멸이 유예되고 계속 생명이 유지되는 변수

1. 클로저의 구동 원리 및 의의

- 작동 기전:
 - 외부 함수가 반환한 내부 함수가 상위 환경의 변수를 계속해서 참조하고 있는 상태
 - 이 경우, 해당 렉시컬 환경은 메모리 참조 계수가 0이 아니므로 가비지 컬렉터(GC)의 수거 대상에서 영구 제외됨
- 실무적 핵심 효용:
 - 상태 은닉 및 캡슐화: 전역 변수를 쓰지 않고 외부에서 함부로 조작할 수 없도록 특정 변수를 프라이빗하게 격리함
 - 부수 효과 차단: 프라이빗 상태 조작 통로를 단 하나의 클로저 함수로 단일화하여 무분별한 조작을 원천 차단함

클로저 상태 은닉 코드 예시

```
1  function createCounter() {  
2      let count = 0; // 외부 함수 내의 프라이빗 상태 (안전하게 은닉 성공)  
3  
4      // 상위 환경인 count=0 을 평생 기억하는 클로저 함수 반환  
5      return function () {  
6          count++;  
7          return count;  
8      };  
9  }  
10  
11 const counter = createCounter();  
12  
13 console.log(counter()); // 1 (상위 환경 count 기억하여 0 -> 1 증가)  
14 console.log(counter()); // 2 (계속 보존되고 있는 1 -> 2 상태 갱신 반영)
```